①

SHORT ANSWER TEST.

NAME: REKHA. SK

REG NO: 19251102 3

COURSE CODE: CSA 1404

COURSE NAME: COMPILER DESIGN

DATE: 1/08/2026

FACULTY: Dr. N. Deepa.

73 / 100

Answers.

1) $E \rightarrow E + E \,/\, id.$

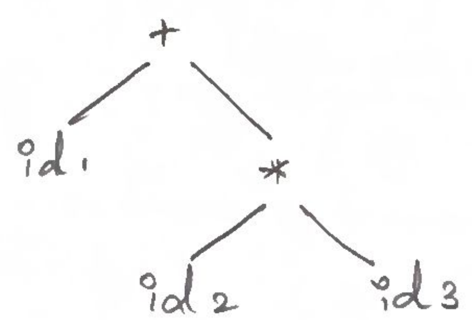
2) a) NFA for regular expression
$a(a/b)^* a.$
Draw state transition diagram

3) Parse tree for the string $id + id * id$

(i) Lexical analysis

<id1> <+> <id2> <*> <id3>

(ii) Syntax analysis (Parse tree).



$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E).$

2

4) Eliminate left recursion $A \rightarrow A\alpha / \beta$

$A \rightarrow A\alpha$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon$

2

5) $S \rightarrow iEtS / iEtSeS / a$ Left factoring.

$S \rightarrow iEtS$

$S \rightarrow iEtSeS$ $S = a$

$S \rightarrow a$

(ii) Left factoring.

~~$S \rightarrow \mathit{eS}$~~

$S \rightarrow iEtSS'$ $S \rightarrow a$.

$S' \rightarrow \mathit{eS}$

3

6) First and Follow

$S \rightarrow aBDh$

$B \rightarrow cC$

$C \rightarrow bC / \epsilon$

$D \rightarrow EF$

$E \rightarrow g / \epsilon$

$F \rightarrow f / \epsilon$

Left recursion is not required.

FIRST (S) = { a }

FIRST (B) = { c }

FIRST (C) = { b, ∈ }

FIRST (D) = { g, ∈ }

FIRST (E) = { g, ∈ }

FIRST (F) = { f, ∈ }

FOLLOW (S) = { $ }

FOLLOW (B) = { $ }

FOLLOW (C) = { $ }

FOLLOW (D) = { $ }

FOLLOW (E) = { f, g, $ }

FOLLOW (F) = { f, g, $ }

S -> aBDh
αBβ
B -> cC
C -> bC/ε
αBβ
D -> EF
αBβ
E -> g/ε
F -> f/ε

7) S -> Sa/Sb/cd

S -> Sa

S -> Sb

S -> cd

This grammer doesn't contains immediate left recursion.

8) E -> TE'

E' -> +TE'/ε

To find: First of (E')

FIRST (E') = { +, ε }

First(E') is { +, ε }.

9) S -> (S) S / ε

~~I~~ The string ((C)) is Invalid. The string S -> S*S/ε is correct and valid one.

10) Actions performed on Shift-Reducing Parsing:

1) Shift

2) Reduce

There are the two actions performed in Shift-Reduce Parsing

11) $S' \to S$

$S \to CC$

$C \to cC / d$

Item: $[S' \to \cdot S]$

Answer.

$S' \to \cdot S$

$S \to \cdot CC$

$C \to \cdot cC / \cdot d$

The closure $S' \to \cdot S$ is found.

12) (i) $S \to AB$

$A \to a$

$B \to b$

(ii) The grammar $S \to aSb / ab$

To check: aabb is accepted

Answer

(i) $S \to AB$

$A \to a$

$B \to b$

$FIRST(S) = \{a\}$

$FIRST(A) = \{a\}$

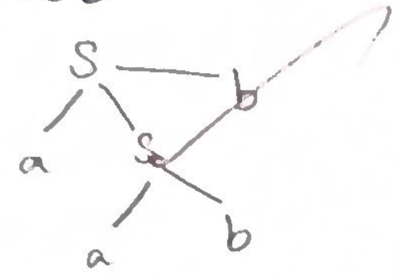
$FIRST(B) = \{b\}$

(ii) $S \to aSb / ab$

~~The~~ To Check: aabb is accepted

$S \rightarrow aSb$

$S \rightarrow ab$



aabb is not accepted.

13) **First and Follow**

$S \rightarrow A$

$A \rightarrow aB / Ad$

$B \rightarrow b$

$C \rightarrow g$

Answer.

1) $S \rightarrow A$
2) $A \rightarrow aB / Ad$

   $A \rightarrow aB$

   $A \rightarrow Ad$

   $A \rightarrow aBA'$

   $A' \rightarrow dA' / \epsilon$

3) $B \rightarrow b$
4) $C \rightarrow g$.

1) $S \rightarrow A$
2) $A \rightarrow aBA'$
3) $A' \rightarrow dA' / \epsilon$
4) $B \rightarrow b$
5) $C \rightarrow g$.

$FIRST(S) = \{a\}$

$FIRST(A) = \{a\}$

$FIRST(A') = \{d, \epsilon\}$

$FIRST(B) = \{b\}$

$FIRST(C) = \{g\}$

FOLLOW(S) = {$}

FOLLOW(A) = {$}

FOLLOW(A') = {$}

FOLLOW(B) = {d, $}

FOLLOW(C) = {d, $}.

S -> ...

A -> αEA'
αEE

A' -> dA'
αBB

B -> b

C -> g.

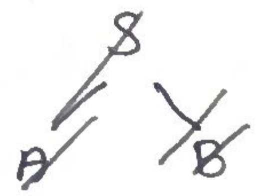
15) S -> AB

A -> aA/a

B -> bB/b

Find LMD and RMD for "aaabb".

Answer.



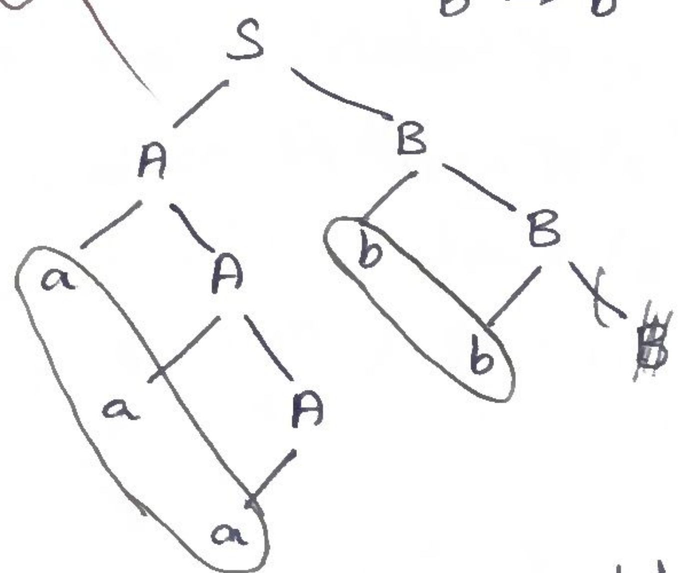
S -> AB

A -> aA

A -> a

B -> bB

B -> b



aaabb.

No performing LMD and RMD we got "aaabb".

16) S -> AB

A -> aA/ε

B -> bB/c

(9).

Answer

$S \to AB$

$A \to aA / \varepsilon$

$B \to bB / c$

~~$B \to CB'$~~

~~$B' \to$~~

$FIRST(S) = \{a, \varepsilon\}$

$FIRST(A) = \{a, \varepsilon\}$

$FIRST(B) = \{b, c\}$

$FOLLOW(S) = \{\$, a\}$

$FOLLOW(A) = \{\$, a\}$

$FOLLOW(B) = \{\$, a\}$.

$S \to AB$
$\alpha \; \beta$

$A \to aA$
$\alpha \; \beta$

$B \to bB$

18) **Input string:** id + id * id

(i) Recursive - Descent (Top - Down) Parsing.

(ii) Shift Reduce (Bottom - up) Parsing

Grammar.

(i) $E \to TE'$

(ii) $E' \to +TE' / \varepsilon$

(iii) $T \to FT'$

(iv) $T' \to *FT' / \varepsilon$

(v) $F \to (E) / id$.

(i) Recursive-Descent (Top-Down)

1) FIRST

FIRST(E) = { (, id }

FIRST(E') = { +, ε }

FIRST(T) = { (, id }

FIRST(T') = { *, ε }

FIRST(F) = { (, id }

FOLLOW

FOLLOW(E) = { $,) }

FOLLOW(E') = { $,) }

FOLLOW ~~$~~(T) = { +, $,) }

FOLLOW(T') = { +, $,) }

FOLLOW(F) = { *, +, $,) }

E -> TE'
α B β

E' -> +TE'
α B β

T -> FT'
α B β

T' -> *FT'
α B β

F -> (E)
α B β

2) Predictive Parsing Table

Non-Terminal	Terminal					
	id	+	*	(	)	$
E	E->TE'			E->TE'		
E'		E'->+TE'			E'->ε	E'->ε
T	T->FT'			T->FT'		
T'		T'->ε	T'->*FT'		T'->ε	T'->ε
F	F->id		F->(E) id			

3) Stack Implementation

Stack	Input	Output.
$\$$	$id + id * id\,\$$	-
$\$E'T$	$id + id * id\,\$$	$E \rightarrow TE'$
$\$E'T'F$	$\cancel{id} + id * id\,\$$	$T \rightarrow FT'$
$\$E'T'id$	$+ id * id\,\$$	$F \rightarrow id$
$\$E'\,\cancel{id}$	$* id * id\,\$$	$T' \rightarrow \varepsilon$
$\$E'T+$	$id * id\,\$$	$E' \rightarrow +TE'$
$\$T*$	$id * id\,\$$	$E' \rightarrow \varepsilon$
$\$T'F$	$\cancel{id} * id\,\$$	$T \rightarrow FT'$
$\$T'\cancel{id}$	$* id\,\$$	$F \rightarrow id$
$\$T'F*$	$id\,\$$	$T' \rightarrow *FT'$
$\$F*$	$\cancel{id}\,\$$	$T' \rightarrow \varepsilon$
$\$\cancel{id}$	$\$$	$F \rightarrow id$
$\$$	$\$$	Accept

10

(10)

$E \to E+E$
$E \to E*E$
$E \to id.$

(ii) Shift Reduce Parsing

Stack	Input Buffer	Output
\$	id + id * id \$	-
\$ id	+ id * id \$	Shift
\$ E	+ id * id \$	Reduce ($E \to id$)
\$ E +	id * id \$	Shift
\$ E + id	* id \$	Shift
\$ E + E	* id \$	Reduce ($E \to id$)
\$ E	* id \$	Reduce ($E \to E+E$)
\$ E *	id \$	Shift
\$ E * id	\$	Shift
\$ E * E	\$	Reduce ($E \to id$)
\$ E	\$	Reduce ($E \to E*E$) Accept.

10

10) Actions performed by Shift Reduce Parsing-

1) Shift
2) Reduce
3) Accept.

Shift => Eg: id * id + id

Shift

id | * id + id.

3

Reduce => Eg: E -> (E).

Accept => Accepting.

17) <u>Left Recursion</u>

<u>Eg</u>: $A \rightarrow A\alpha / \beta$

$A \rightarrow A\alpha$

$A \rightarrow \beta A'$

3

$A' \rightarrow \alpha A' / \epsilon$.

~~If there is not same noterminal~~

If there is no same non-terminal present on the right side Left recursion is performed.

Left factoring

Eg:.

S -> iEtS

S -> iEtSeS

S -> a

S -> iEtS S'

S' -> leS

S = a

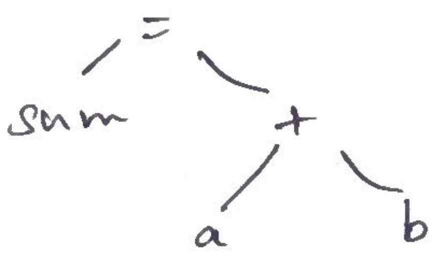
S -> a.

20) a) Lexical Analysis.

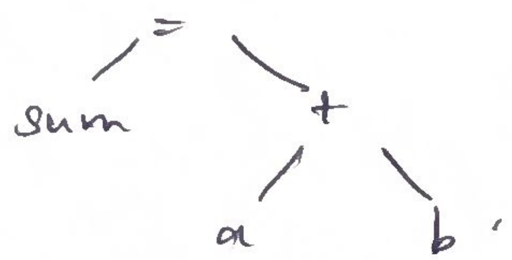
<a><=><10> <b><=><20>

<sum> =<a><+><b>

b) Syntax Analysis (Parse tree)



c) Semantic Analysis



d) Intermediate

$T_1 = \text{int to real } (10)$

T_2 int to real (20)

$T_3 = T_1 + T_2$

$T_4 = T_3$

e) Code Optimization

$T_1 = \text{int to real } (10)(20)$

$T_2 = \cancel{E+t}\ a + b$

$T_3 = T_2$

f) Code Gen

```
LOAD  R1, 10
LOAD  R2, 20
ADD   R2, R1, R2
LOAD  R3
STF   R3, R2
```

8